Student Name: Gyan Tamang

Student ID: 42195

Course Title: Algorithms and Data Structures

Assignment: Divide and Conquer Algorithms Analysis

Date: March 2026

1. Asymptotic Analysis and Recurrence Relations

1.1 Quick Sort

Problem Description

Quick Sort is a divide-and-conquer sorting algorithm used to sort a list of elements. It works by selecting a pivot element, partitioning the array into two subarrays: elements smaller than the pivot go to the left, and elements larger than the pivot go to the right, and then recursively sorting the subarrays.

Key Steps

Choose a pivot element from the array.

Partition the array so that:

Elements smaller than the pivot are on the left.

Elements larger than the pivot are on the right.

Recursively apply Quick Sort to the left and right partitions.

Combine the results to obtain a sorted array.

Time Complexity Analysis

Case	Complexity
Best Case	$\Omega(n \log n)$
Average Case	$\Theta(n \log n)$
Worst Case	$O(n^2)$

Best Case: Occurs when the pivot divides the array into two halves.

Average Case: Most practical cases produce near-balanced partitions.

Worst Case: Occurs when the pivot is always the smallest or largest element (e.g., already sorted array).

Recurrence Relation

Average Case: $T(n) = 2T(n/2) + n$

Worst Case: $T(n) = T(n-1) + n$

Solving Recurrence

Substitution Method

Assume: $T(n) \leq cn \log n$

Substituting into the recurrence confirms that the time complexity is $O(n \log n)$ for balanced partitions.

Recursion Tree Method

Each level of recursion performs $O(n)$ work during partitioning.

Depth of recursion $\approx \log n$

Total cost:

$n + n + n + \ldots$ (log n times)

$= O(n \log n)$

Master Method

Given:

$T(n) = 2T(n/2) + n$

Where: $a = 2$, $b = 2$, $f(n) = n$

$n^{(\log_b a)} = n^{(\log_2 2)} = n$

Since $f(n) = \Theta(n)$, it falls under Case 2 of Master Theorem

Therefore: $T(n) = \Theta(n \log n)$

Practical Implications

Quick Sort is widely used because of its excellent average performance and cache efficiency. Many standard libraries implement optimized versions of Quick Sort. However, its worst-case complexity can be problematic if

pivot selection is poor. Techniques such as randomized pivot selection or the median-of-three pivot are often used to avoid worst-case scenarios.

1.2 Merge Sort

Problem Description

Merge Sort is another divide-and-conquer sorting algorithm that divides the array into two halves, recursively sorts each half, and then merges the sorted halves back together.

Key Steps

Divide the array into two halves.

Recursively sort both halves.

Merge the sorted halves into a single sorted array.

Time Complexity Analysis

Case	Complexity
Best Case	$\Omega(n \log n)$
Average Case	$\Theta(n \log n)$
Worst Case	$O(n \log n)$

Unlike Quick Sort, Merge Sort maintains consistent performance across all cases.

Recurrence Relation

$T(n) = 2T(n/2) + n$

Where: $2T(n/2) \rightarrow$ recursive sorting, $n \rightarrow$ merging cost

Solving Recurrence

Substitution Method

Assume: $T(n) \leq cn \log n$

Substituting confirms the recurrence satisfies $O(n \log n)$.

Recursion Tree Method

Level 1 → n

Level 2 → n

Level 3 → n

Number of levels = log n

Total cost: n × log n = O(n log n)

Master Method

Given:

T(n) = 2T(n/2) + n

Where: a = 2, b = 2, f(n) = n

Since

n^(log_b a) = n

Using Master Theorem Case 2

T(n) = Θ(n log n)

Practical Implications

Merge Sort is highly efficient for large datasets and external sorting, such as sorting files stored on disk. It is also stable, meaning it preserves the relative order of equal elements. However, it requires additional O(n) space, making it less space-efficient than Quick Sort.

2. Implementation and Comparison

Python Implementation

Quick Sort

```
"""
QuickSort Implementation - MSCS532 Assignment 2
Author: Gyan Tamang
Date: March 2026

This module implements various QuickSort algorithms, including:
1. Basic QuickSort (extra space)
2. In-place QuickSort
3. Randomized QuickSort
4. QuickSort with different pivot strategies
```

```python
"""

import random
import time
import sys
from typing import List, Callable

# Increase recursion limit for large arrays
sys.setrecursionlimit(10000)


def quick_sort_basic(arr: List[int]) -> List[int]:
    """
    Basic QuickSort implementation using extra space.

    Time Complexity: O(n log n) average, O(n^2) worst case
    Space Complexity: O(n) due to creating new lists

    Args:
        arr: List of integers to sort

    Returns:
        Sorted list of integers
    """
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]

    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort_basic(left) + middle + quick_sort_basic(right)


def partition(arr: List[int], low: int, high: int) -> int:
    """
    Partition function for in-place QuickSort using Lomuto partition scheme.

    Args:
        arr: List to partition
        low: Starting index
        high: Ending index
```

```
    Returns:
        Final position of pivot element
    """
    pivot = arr[high]
    i = low - 1

    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1


def quick_sort_inplace(arr: List[int], low: int = 0, high: int = None) -> None:
    """
    In-place QuickSort implementation.

    Time Complexity: O(n log n) average, O(n^2) worst case
    Space Complexity: O(log n) due to the recursion stack

    Args:
        arr: List of integers to sort (modified in-place)
        low: Starting index (default 0)
        high: Ending index (default len(arr) - 1)
    """
    if high is None:
        high = len(arr) - 1

    if low < high:
        pi = partition(arr, low, high)
        quick_sort_inplace(arr, low, pi - 1)
        quick_sort_inplace(arr, pi + 1, high)


def partition_randomized(arr: List[int], low: int, high: int) -> int:
    """
    Randomized partition function that selects a random pivot.

    Args:
        arr: List to partition
        low: Starting index
```

```
        high: Ending index

    Returns:
        Final position of pivot element
    """

    random_index = random.randint(low, high)
    arr[random_index], arr[high] = arr[high], arr[random_index]

    return partition(arr, low, high)


def quick_sort_randomized(arr: List[int], low: int = 0, high: int = None) -> None:
    """
    Randomized QuickSort implementation.

    Time Complexity: O(n log n) expected for all cases
    Space Complexity: O(log n) due to the recursion stack

    Args:
        arr: List of integers to sort (modified in-place)
        low: Starting index (default 0)
        high: Ending index (default len(arr) - 1)
    """
    if high is None:
        high = len(arr) - 1

    if low < high:
        pi = partition_randomized(arr, low, high)
        quick_sort_randomized(arr, low, pi - 1)
        quick_sort_randomized(arr, pi + 1, high)


def partition_median_of_three(arr: List[int], low: int, high: int) -> int:
    """
    Partition using median-of-three pivot selection.

    Args:
        arr: List to partition
        low: Starting index
        high: Ending index

    Returns:
        Final position of pivot element
```

```python
    """
    mid = (low + high) // 2

    # Find median of arr[low], arr[mid], arr[high]
    if arr[low] > arr[mid]:
        arr[low], arr[mid] = arr[mid], arr[low]
    if arr[low] > arr[high]:
        arr[low], arr[high] = arr[high], arr[low]
    if arr[mid] > arr[high]:
        arr[mid], arr[high] = arr[high], arr[mid]

    # Place median at second-to-last position
    arr[mid], arr[high - 1] = arr[high - 1], arr[mid]

    pivot = arr[high - 1]
    i = low

    for j in range(low, high - 1):
        if arr[j] <= pivot:
            arr[i], arr[j] = arr[j], arr[i]
            i += 1

    arr[i], arr[high - 1] = arr[high - 1], arr[i]
    return i


def quick_sort_median_of_three(arr: List[int], low: int = 0, high: int = None) ->
None:
    """
    QuickSort with median-of-three pivot selection.

    Time Complexity: O(n log n) average, better than standard pivot
    Space Complexity: O(log n) due to the recursion stack

    Args:
        arr: List of integers to sort (modified in-place)
        low: Starting index (default 0)
        high: Ending index (default len(arr) - 1)
    """
    if high is None:
        high = len(arr) - 1

    if low < high and high - low >= 2:
        pi = partition_median_of_three(arr, low, high)
```

```python
        quick_sort_median_of_three(arr, low, pi - 1)
        quick_sort_median_of_three(arr, pi + 1, high)
    elif low < high:
        # Use simple partition for small subarrays
        pi = partition(arr, low, high)
        quick_sort_median_of_three(arr, low, pi - 1)
        quick_sort_median_of_three(arr, pi + 1, high)


def measure_performance(sort_func: Callable, arr: List[int], name: str) -> tuple:
    """
    Measure the performance of a sorting function.

    Args:
        sort_func: Sorting function to test
        arr: List to sort
        name: Name of the sorting algorithm

    Returns:
        Tuple of (execution_time, sorted_array)
    """
    arr_copy = arr.copy()
    start_time = time.perf_counter()

    if name == "basic":
        result = sort_func(arr_copy)
    else:
        sort_func(arr_copy)
        result = arr_copy

    end_time = time.perf_counter()
    execution_time = (end_time - start_time) * 1000  # Convert to milliseconds

    return execution_time, result


def verify_sorted(arr: List[int]) -> bool:
    """
    Verify if an array is sorted in ascending order.

    Args:
        arr: List to verify

    Returns:
```

```
        True if sorted, False otherwise
    """
    return all(arr[i] <= arr[i + 1] for i in range(len(arr) - 1))


def run_tests():
    """
    Run comprehensive tests on all QuickSort implementations.
    """
    print("=" * 80)
    print("QuickSort Implementation Tests - MSCS532 Assignment 2")
    print("=" * 80)

    # Test Case 1: Already sorted array
    print("\n1. Test Case: Already Sorted Array")
    test1 = list(range(1000))
    print(f"   Input size: {len(test1)}")

    time1, sorted1 = measure_performance(quick_sort_basic, test1, "basic")
    print(f"   Basic QuickSort: {time1:.3f} ms | Correct: {verify_sorted(sorted1)}")

    time2, sorted2 = measure_performance(quick_sort_inplace, test1, "inplace")
    print(f"   In-place QuickSort: {time2:.3f} ms | Correct:
{verify_sorted(sorted2)}")

    time3, sorted3 = measure_performance(quick_sort_randomized, test1, "randomized")
    print(f"   Randomized QuickSort: {time3:.3f} ms | Correct:
{verify_sorted(sorted3)}")

    time4, sorted4 = measure_performance(quick_sort_median_of_three, test1, "median")
    print(f"   Median-of-Three QuickSort: {time4:.3f} ms | Correct:
{verify_sorted(sorted4)}")

    # Test Case 2: Reverse sorted array
    print("\n2. Test Case: Reverse Sorted Array")
    test2 = list(range(1000, 0, -1))
    print(f"   Input size: {len(test2)}")

    time1, sorted1 = measure_performance(quick_sort_basic, test2, "basic")
    print(f"   Basic QuickSort: {time1:.3f} ms | Correct: {verify_sorted(sorted1)}")

    time2, sorted2 = measure_performance(quick_sort_inplace, test2, "inplace")
    print(f"   In-place QuickSort: {time2:.3f} ms | Correct:
{verify_sorted(sorted2)}")
```

```
    time3, sorted3 = measure_performance(quick_sort_randomized, test2, "randomized")
    print(f"   Randomized QuickSort: {time3:.3f} ms | Correct:
{verify_sorted(sorted3)}")

    time4, sorted4 = measure_performance(quick_sort_median_of_three, test2, "median")
    print(f"   Median-of-Three QuickSort: {time4:.3f} ms | Correct:
{verify_sorted(sorted4)}")

    # Test Case 3: Random array
    print("\n3. Test Case: Random Array")
    test3 = [random.randint(1, 10000) for _ in range(1000)]
    print(f"   Input size: {len(test3)}")

    time1, sorted1 = measure_performance(quick_sort_basic, test3, "basic")
    print(f"   Basic QuickSort: {time1:.3f} ms | Correct: {verify_sorted(sorted1)}")

    time2, sorted2 = measure_performance(quick_sort_inplace, test3, "inplace")
    print(f"   In-place QuickSort: {time2:.3f} ms | Correct:
{verify_sorted(sorted2)}")

    time3, sorted3 = measure_performance(quick_sort_randomized, test3, "randomized")
    print(f"   Randomized QuickSort: {time3:.3f} ms | Correct:
{verify_sorted(sorted3)}")

    time4, sorted4 = measure_performance(quick_sort_median_of_three, test3, "median")
    print(f"   Median-of-Three QuickSort: {time4:.3f} ms | Correct:
{verify_sorted(sorted4)}")

    # Test Case 4: Array with duplicates
    print("\n4. Test Case: Array with Many Duplicates")
    test4 = [random.randint(1, 10) for _ in range(1000)]
    print(f"   Input size: {len(test4)}")

    time1, sorted1 = measure_performance(quick_sort_basic, test4, "basic")
    print(f"   Basic QuickSort: {time1:.3f} ms | Correct: {verify_sorted(sorted1)}")

    time2, sorted2 = measure_performance(quick_sort_inplace, test4, "inplace")
    print(f"   In-place QuickSort: {time2:.3f} ms | Correct:
{verify_sorted(sorted2)}")

    time3, sorted3 = measure_performance(quick_sort_randomized, test4, "randomized")
    print(f"   Randomized QuickSort: {time3:.3f} ms | Correct:
{verify_sorted(sorted3)}")
```

```
    time4, sorted4 = measure_performance(quick_sort_median_of_three, test4, "median")
    print(f"   Median-of-Three QuickSort: {time4:.3f} ms | Correct:
{verify_sorted(sorted4)}")

    # Test Case 5: Small arrays
    print("\n5. Test Case: Small Arrays")
    test5_cases = [
        [],
        [1],
        [2, 1],
        [3, 1, 2],
        [5, 2, 8, 1, 9]
    ]

    for i, test5 in enumerate(test5_cases):
        print(f"   Test 5.{i+1}: {test5}")
        sorted_result = quick_sort_basic(test5.copy())
        print(f"   Result: {sorted_result} | Correct:
{verify_sorted(sorted_result)}")

    # Performance Comparison
    print("\n" + "=" * 80)
    print("Performance Comparison on Large Random Array (5000 elements)")
    print("=" * 80)

    large_test = [random.randint(1, 100000) for _ in range(5000)]

    algorithms = [
        ("Basic QuickSort", quick_sort_basic, "basic"),
        ("In-place QuickSort", quick_sort_inplace, "inplace"),
        ("Randomized QuickSort", quick_sort_randomized, "randomized"),
        ("Median-of-Three QuickSort", quick_sort_median_of_three, "median")
    ]

    results = []
    for name, func, func_type in algorithms:
        exec_time, sorted_arr = measure_performance(func, large_test, func_type)
        results.append((name, exec_time, verify_sorted(sorted_arr)))
        print(f"{name:30s}: {exec_time:8.3f} ms | Correct:
{verify_sorted(sorted_arr)}")

    print("\n" + "=" * 80)
    print("Analysis Summary:")
```

```
    print("=" * 80)
    print("""
Time Complexity:
- Best Case:    O(n log n) - when partition divides array evenly
- Average Case: O(n log n) - expected performance with random pivots
- Worst Case:   O(n²) - when partition is unbalanced (sorted/reverse sorted)

Space Complexity:
- Basic QuickSort:     O(n) - creates new lists at each recursion
- In-place QuickSort:  O(log n) - recursion stack only
- Randomized:          O(log n) - recursion stack only
- Median-of-Three:     O(log n) - recursion stack only

Key Observations:
1. Randomized QuickSort performs well on all input types
2. Median-of-Three strategy improves performance on sorted data
3. In-place versions use less memory than the basic implementation
4. All versions correctly handle duplicates and edge cases
    """)


if __name__ == "__main__":
    run_tests()
```

Output

==========

QuickSort Implementation Tests - MSCS532 Assignment 2

==========

1. Test Case: Already Sorted Array

   Input size: 1000

   Basic QuickSort: 0.983 ms | Correct: True

   In-place QuickSort: 91.715 ms | Correct: True

   Randomized QuickSort: 1.556 ms | Correct: True

   Median-of-Three QuickSort: 1.070 ms | Correct: True

2. Test Case: Reverse Sorted Array

Input size: 1000

Basic QuickSort: 1.298 ms | Correct: True

In-place QuickSort: 38.592 ms | Correct: True

Randomized QuickSort: 1.665 ms | Correct: True

Median-of-Three QuickSort: 1.110 ms | Correct: True

3. Test Case: Random Array

Input size: 1000

Basic QuickSort: 1.574 ms | Correct: True

In-place QuickSort: 1.181 ms | Correct: True

Randomized QuickSort: 1.704 ms | Correct: True

Median-of-Three QuickSort: 1.316 ms | Correct: True

4. Test Case: Array with Many Duplicates

Input size: 1000

Basic QuickSort: 0.665 ms | Correct: True

In-place QuickSort: 6.050 ms | Correct: True

Randomized QuickSort: 7.096 ms | Correct: True

Median-of-Three QuickSort: 3.244 ms | Correct: True

5. Test Case: Small Arrays

Test 5.1: []

Result: [] | Correct: True

Test 5.2: [1]

Result: [1] | Correct: True

Test 5.3: [2, 1]

Result: [1, 2] | Correct: True

Test 5.4: [3, 1, 2]

Result: [1, 2, 3] | Correct: True

Test 5.5: [5, 2, 8, 1, 9]

Result: [1, 2, 5, 8, 9] | Correct: True

==========

Performance Comparison on Large Random Array (5000 elements)

==========

Basic QuickSort : 10.445 ms | Correct: True

In-place QuickSort : 7.305 ms | Correct: True

Randomized QuickSort : 15.050 ms | Correct: True

Median-of-Three QuickSort : 11.595 ms | Correct: True

==========

Analysis Summary:

==========

Time Complexity:

- Best Case: O(n log n) - when partition divides array evenly

- Average Case: O(n log n) - expected performance with random pivots

- Worst Case: $O(n^2)$ - when partition is unbalanced (sorted/reverse sorted)

Space Complexity:

- Basic QuickSort: O(n) - creates new lists at each recursion

- In-place QuickSort: O(log n) - recursion stack only

- Randomized: O(log n) - recursion stack only

- Median-of-Three: O(log n) - recursion stack only

Key Observations:

1. Randomized QuickSort performs well on all input types

2. Median-of-Three strategy improves performance on sorted data

3. In-place versions use less memory than the basic implementation

4. All versions correctly handle duplicates and edge cases

Merge Sort

```
"""
MergeSort Implementation - MSCS532 Assignment 2
Author: Gyan Tamang
Date: March 2026

This module implements various MergeSort algorithms, including:
1. Basic MergeSort (extra space)
2. In-place MergeSort
3. Optimized MergeSort with insertion sort for small subarrays
4. Bottom-up (iterative) MergeSort
"""

import random
import time
import sys
from typing import List, Callable

# Increase recursion limit for large arrays
sys.setrecursionlimit(10000)


def merge(left: List[int], right: List[int]) -> List[int]:
    """
    Merge two sorted arrays into a single sorted array.

    Time Complexity: O(n + m), where n and m are the lengths of the left and right
    Space Complexity: O(n + m) for result array

    Args:
```

```
        left: First sorted list
        right: Second sorted list

    Returns:
        Merged sorted list
    """
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result


def merge_sort_basic(arr: List[int]) -> List[int]:
    """
    Basic MergeSort implementation using extra space.

    Time Complexity: O(n log n) for all cases
    Space Complexity: O(n) for temporary arrays

    Args:
        arr: List of integers to sort

    Returns:
        Sorted list of integers
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort_basic(arr[:mid])
    right = merge_sort_basic(arr[mid:])
```

```python
    return merge(left, right)


def merge_inplace(arr: List[int], left: int, mid: int, right: int) -> None:
    """
    Merge two sorted subarrays in-place.

    Args:
        arr: Array containing the subarrays
        left: Starting index of left subarray
        mid: Ending index of left subarray
        right: Ending index of right subarray
    """
    # Create temporary arrays
    left_arr = arr[left:mid + 1]
    right_arr = arr[mid + 1:right + 1]

    i = j = 0
    k = left

    while i < len(left_arr) and j < len(right_arr):
        if left_arr[i] <= right_arr[j]:
            arr[k] = left_arr[i]
            i += 1
        else:
            arr[k] = right_arr[j]
            j += 1
        k += 1

    while i < len(left_arr):
        arr[k] = left_arr[i]
        i += 1
        k += 1

    while j < len(right_arr):
        arr[k] = right_arr[j]
        j += 1
        k += 1


def merge_sort_inplace(arr: List[int], left: int = 0, right: int = None) -> None:
    """
    In-place MergeSort implementation.
```

```
    Time Complexity: O(n log n) for all cases
    Space Complexity: O(n) for temporary arrays during merge

    Args:
        arr: List of integers to sort (modified in-place)
        left: Starting index (default 0)
        right: Ending index (default len(arr) - 1)
    """
    if right is None:
        right = len(arr) - 1

    if left < right:
        mid = (left + right) // 2

        merge_sort_inplace(arr, left, mid)
        merge_sort_inplace(arr, mid + 1, right)

        merge_inplace(arr, left, mid, right)


def insertion_sort(arr: List[int], left: int, right: int) -> None:
    """
    Insertion sort for small subarrays.

    Time Complexity: O(n²) but fast for small n
    Space Complexity: O(1)

    Args:
        arr: Array to sort
        left: Starting index
        right: Ending index
    """
    for i in range(left + 1, right + 1):
        key = arr[i]
        j = i - 1
        while j >= left and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key


def merge_sort_optimized(arr: List[int], left: int = 0, right: int = None, threshold:
int = 10) -> None:
    """
```

```
    Optimized MergeSort that uses insertion sort for small subarrays.

    Time Complexity: O(n log n) for all cases
    Space Complexity: O(n) for temporary arrays

    Args:
        arr: List of integers to sort (modified in-place)
        left: Starting index (default 0)
        right: Ending index (default len(arr) - 1)
        threshold: Size threshold for switching to insertion sort
    """
    if right is None:
        right = len(arr) - 1

    if left < right:
        # Use insertion sort for small subarrays
        if right - left + 1 <= threshold:
            insertion_sort(arr, left, right)
        else:
            mid = (left + right) // 2

            merge_sort_optimized(arr, left, mid, threshold)
            merge_sort_optimized(arr, mid + 1, right, threshold)

            # Skip merge if already sorted
            if arr[mid] <= arr[mid + 1]:
                return

            merge_inplace(arr, left, mid, right)


def merge_sort_iterative(arr: List[int]) -> None:
    """
    Bottom-up iterative MergeSort implementation.

    Time Complexity: O(n log n) for all cases
    Space Complexity: O(n) for temporary arrays

    Args:
        arr: List of integers to sort (modified in-place)
    """
    n = len(arr)
    current_size = 1
```

```python
    while current_size < n:
        left = 0

        while left < n:
            mid = min(left + current_size - 1, n - 1)
            right = min(left + 2 * current_size - 1, n - 1)

            if mid < right:
                merge_inplace(arr, left, mid, right)

            left += 2 * current_size

        current_size *= 2


def merge_sort_three_way(arr: List[int]) -> List[int]:
    """
    Three-way MergeSort that divides the array into three parts.

    Time Complexity: O(n log₃ n) ≈ O(n log n)
    Space Complexity: O(n)

    Args:
        arr: List of integers to sort

    Returns:
        Sorted list of integers
    """
    if len(arr) <= 1:
        return arr

    # For small arrays, use two-way merge sort to avoid recursion issues
    if len(arr) == 2:
        return [arr[0], arr[1]] if arr[0] <= arr[1] else [arr[1], arr[0]]

    # Divide into three parts
    third = len(arr) // 3

    # Ensure we make progress by having at least one element in each split
    if third == 0:
        third = 1

    left = merge_sort_three_way(arr[:third])
    middle = merge_sort_three_way(arr[third:2*third])
```

```python
    right = merge_sort_three_way(arr[2*third:])

    # Merge three sorted arrays
    return merge_three(left, middle, right)


def merge_three(left: List[int], middle: List[int], right: List[int]) -> List[int]:
    """
    Merge three sorted arrays into one.

    Args:
        left: First sorted list
        middle: Second sorted list
        right: Third sorted list

    Returns:
        Merged sorted list
    """
    result = []
    i = j = k = 0

    while i < len(left) and j < len(middle) and k < len(right):
        if left[i] <= middle[j] and left[i] <= right[k]:
            result.append(left[i])
            i += 1
        elif middle[j] <= left[i] and middle[j] <= right[k]:
            result.append(middle[j])
            j += 1
        else:
            result.append(right[k])
            k += 1

    # Merge the remaining two arrays
    while i < len(left) and j < len(middle):
        if left[i] <= middle[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(middle[j])
            j += 1

    while i < len(left) and k < len(right):
        if left[i] <= right[k]:
            result.append(left[i])
```

```
            i += 1
        else:
            result.append(right[k])
            k += 1

    while j < len(middle) and k < len(right):
        if middle[j] <= right[k]:
            result.append(middle[j])
            j += 1
        else:
            result.append(right[k])
            k += 1

    # Add the remaining elements'
    result.extend(left[i:])
    result.extend(middle[j:])
    result.extend(right[k:])

    return result


def measure_performance(sort_func: Callable, arr: List[int], name: str) -> tuple:
    """
    Measure the performance of a sorting function.

    Args:
        sort_func: Sorting function to test
        arr: List to sort
        name: Name of the sorting algorithm

    Returns:
        Tuple of (execution_time, sorted_array)
    """
    arr_copy = arr.copy()
    start_time = time.perf_counter()

    if name in ["basic", "three_way"]:
        result = sort_func(arr_copy)
    else:
        sort_func(arr_copy)
        result = arr_copy

    end_time = time.perf_counter()
    execution_time = (end_time - start_time) * 1000  # Convert to milliseconds
```

```python
    return execution_time, result


def verify_sorted(arr: List[int]) -> bool:
    """
    Verify if an array is sorted in ascending order.

    Args:
        arr: List to verify

    Returns:
        True if sorted, False otherwise
    """
    return all(arr[i] <= arr[i + 1] for i in range(len(arr) - 1))


def run_tests():
    """
    Run comprehensive tests on all MergeSort implementations.
    """
    print("=" * 80)
    print("MergeSort Implementation Tests - MSCS532 Assignment 2")
    print("=" * 80)

    # Test Case 1: Already sorted array
    print("\n1. Test Case: Already Sorted Array")
    test1 = list(range(1000))
    print(f"   Input size: {len(test1)}")

    time1, sorted1 = measure_performance(merge_sort_basic, test1, "basic")
    print(f"   Basic MergeSort: {time1:.3f} ms | Correct: {verify_sorted(sorted1)}")

    time2, sorted2 = measure_performance(merge_sort_inplace, test1, "inplace")
    print(f"   In-place MergeSort: {time2:.3f} ms | Correct:
{verify_sorted(sorted2)}")

    time3, sorted3 = measure_performance(merge_sort_optimized, test1, "optimized")
    print(f"   Optimized MergeSort: {time3:.3f} ms | Correct:
{verify_sorted(sorted3)}")

    time4, sorted4 = measure_performance(merge_sort_iterative, test1, "iterative")
    print(f"   Iterative MergeSort: {time4:.3f} ms | Correct:
{verify_sorted(sorted4)}")
```

```
    time5, sorted5 = measure_performance(merge_sort_three_way, test1, "three_way")
    print(f"   Three-way MergeSort: {time5:.3f} ms | Correct:
{verify_sorted(sorted5)}")

    # Test Case 2: Reverse sorted array
    print("\n2. Test Case: Reverse Sorted Array")
    test2 = list(range(1000, 0, -1))
    print(f"   Input size: {len(test2)}")

    time1, sorted1 = measure_performance(merge_sort_basic, test2, "basic")
    print(f"   Basic MergeSort: {time1:.3f} ms | Correct: {verify_sorted(sorted1)}")

    time2, sorted2 = measure_performance(merge_sort_inplace, test2, "inplace")
    print(f"   In-place MergeSort: {time2:.3f} ms | Correct:
{verify_sorted(sorted2)}")

    time3, sorted3 = measure_performance(merge_sort_optimized, test2, "optimized")
    print(f"   Optimized MergeSort: {time3:.3f} ms | Correct:
{verify_sorted(sorted3)}")

    time4, sorted4 = measure_performance(merge_sort_iterative, test2, "iterative")
    print(f"   Iterative MergeSort: {time4:.3f} ms | Correct:
{verify_sorted(sorted4)}")

    time5, sorted5 = measure_performance(merge_sort_three_way, test2, "three_way")
    print(f"   Three-way MergeSort: {time5:.3f} ms | Correct:
{verify_sorted(sorted5)}")

    # Test Case 3: Random array
    print("\n3. Test Case: Random Array")
    test3 = [random.randint(1, 10000) for _ in range(1000)]
    print(f"   Input size: {len(test3)}")

    time1, sorted1 = measure_performance(merge_sort_basic, test3, "basic")
    print(f"   Basic MergeSort: {time1:.3f} ms | Correct: {verify_sorted(sorted1)}")

    time2, sorted2 = measure_performance(merge_sort_inplace, test3, "inplace")
    print(f"   In-place MergeSort: {time2:.3f} ms | Correct:
{verify_sorted(sorted2)}")

    time3, sorted3 = measure_performance(merge_sort_optimized, test3, "optimized")
    print(f"   Optimized MergeSort: {time3:.3f} ms | Correct:
{verify_sorted(sorted3)}")
```

```
    time4, sorted4 = measure_performance(merge_sort_iterative, test3, "iterative")
    print(f"   Iterative MergeSort: {time4:.3f} ms | Correct:
{verify_sorted(sorted4)}")

    time5, sorted5 = measure_performance(merge_sort_three_way, test3, "three_way")
    print(f"   Three-way MergeSort: {time5:.3f} ms | Correct:
{verify_sorted(sorted5)}")

    # Test Case 4: Array with duplicates
    print("\n4. Test Case: Array with Many Duplicates")
    test4 = [random.randint(1, 10) for _ in range(1000)]
    print(f"   Input size: {len(test4)}")

    time1, sorted1 = measure_performance(merge_sort_basic, test4, "basic")
    print(f"   Basic MergeSort: {time1:.3f} ms | Correct: {verify_sorted(sorted1)}")

    time2, sorted2 = measure_performance(merge_sort_inplace, test4, "inplace")
    print(f"   In-place MergeSort: {time2:.3f} ms | Correct:
{verify_sorted(sorted2)}")

    time3, sorted3 = measure_performance(merge_sort_optimized, test4, "optimized")
    print(f"   Optimized MergeSort: {time3:.3f} ms | Correct:
{verify_sorted(sorted3)}")

    time4, sorted4 = measure_performance(merge_sort_iterative, test4, "iterative")
    print(f"   Iterative MergeSort: {time4:.3f} ms | Correct:
{verify_sorted(sorted4)}")

    time5, sorted5 = measure_performance(merge_sort_three_way, test4, "three_way")
    print(f"   Three-way MergeSort: {time5:.3f} ms | Correct:
{verify_sorted(sorted5)}")

    # Test Case 5: Small arrays
    print("\n5. Test Case: Small Arrays")
    test5_cases = [
        [],
        [1],
        [2, 1],
        [3, 1, 2],
        [5, 2, 8, 1, 9]
    ]

    for i, test5 in enumerate(test5_cases):
```

```
        print(f"   Test 5.{i+1}: {test5}")
        sorted_result = merge_sort_basic(test5.copy())
        print(f"   Result: {sorted_result} | Correct:
{verify_sorted(sorted_result)}")

    # Performance Comparison
    print("\n" + "=" * 80)
    print("Performance Comparison on Large Random Array (5000 elements)")
    print("=" * 80)

    large_test = [random.randint(1, 100000) for _ in range(5000)]

    algorithms = [
        ("Basic MergeSort", merge_sort_basic, "basic"),
        ("In-place MergeSort", merge_sort_inplace, "inplace"),
        ("Optimized MergeSort", merge_sort_optimized, "optimized"),
        ("Iterative MergeSort", merge_sort_iterative, "iterative"),
        ("Three-way MergeSort", merge_sort_three_way, "three_way")
    ]

    results = []
    for name, func, func_type in algorithms:
        exec_time, sorted_arr = measure_performance(func, large_test, func_type)
        results.append((name, exec_time, verify_sorted(sorted_arr)))
        print(f"{name:30s}: {exec_time:8.3f} ms | Correct:
{verify_sorted(sorted_arr)}")

    print("\n" + "=" * 80)
    print("Analysis Summary:")
    print("=" * 80)
    print("""
Time Complexity:
- Best Case:    O(n log n) - consistent performance regardless of input
- Average Case: O(n log n) - always divides array evenly
- Worst Case:   O(n log n) - no degradation like QuickSort

Space Complexity:
- Basic MergeSort:     O(n) - creates new lists at each level
- In-place MergeSort:  O(n) - temporary arrays during merge
- Optimized MergeSort: O(n) - same as in-place but faster
- Iterative MergeSort: O(n) - no recursion stack needed
- Three-way MergeSort: O(n) - similar to basic version

Key Observations:
```

```
1. MergeSort guarantees O(n log n) time complexity in all cases
2. Stable sorting algorithm - preserves the relative order of equal elements
3. Optimized version with insertion sort improves performance on small arrays
4. The iterative version avoids the recursion overhead
5. Three-way version reduces the depth of the recursion tree
6. Better than QuickSort for linked lists and guaranteed performance
7. Requires additional space, unlike in-place QuickSort

Advantages over QuickSort:
- Guaranteed O(n log n) performance (no worst case O(n²))
- Stable sorting (maintains relative order of equal elements)
- Predictable performance for worst-case scenarios
- Better for external sorting (sorting data that doesn't fit in memory)

Disadvantages compared to QuickSort:
- Requires O(n) extra space
- Slower than QuickSort on average for arrays in memory
- More complex to implement a true in-place version
    """)


if __name__ == "__main__":
    run_tests()
```

Output

==========

MergeSort Implementation Tests - MSCS532 Assignment 2

==========

1. Test Case: Already Sorted Array

   Input size: 1000

   Basic MergeSort: 1.445 ms | Correct: True

   In-place MergeSort: 2.226 ms | Correct: True

   Optimized MergeSort: 0.172 ms | Correct: True

   Iterative MergeSort: 1.727 ms | Correct: True

   Three-way MergeSort: 1.085 ms | Correct: True

2. Test Case: Reverse Sorted Array

Input size: 1000

Basic MergeSort: 1.482 ms | Correct: True

In-place MergeSort: 1.797 ms | Correct: True

Optimized MergeSort: 1.458 ms | Correct: True

Iterative MergeSort: 1.828 ms | Correct: True

Three-way MergeSort: 1.184 ms | Correct: True

3. Test Case: Random Array

Input size: 1000

Basic MergeSort: 3.450 ms | Correct: True

In-place MergeSort: 2.621 ms | Correct: True

Optimized MergeSort: 3.066 ms | Correct: True

Iterative MergeSort: 2.453 ms | Correct: True

Three-way MergeSort: 2.299 ms | Correct: True

4. Test Case: Array with Many Duplicates

Input size: 1000

Basic MergeSort: 2.238 ms | Correct: True

In-place MergeSort: 2.218 ms | Correct: True

Optimized MergeSort: 2.758 ms | Correct: True

Iterative MergeSort: 2.586 ms | Correct: True

Three-way MergeSort: 2.436 ms | Correct: True

5. Test Case: Small Arrays

Test 5.1: []

Result: [] | Correct: True

Test 5.2: [1]

Result: [1] | Correct: True

Test 5.3: [2, 1]

Result: [1, 2] | Correct: True

Test 5.4: [3, 1, 2]

Result: [1, 2, 3] | Correct: True

Test 5.5: [5, 2, 8, 1, 9]

Result: [1, 2, 5, 8, 9] | Correct: True

==========

Performance Comparison on Large Random Array (5000 elements)

==========

Basic MergeSort : 24.574 ms | Correct: True

In-place MergeSort : 16.155 ms | Correct: True

Optimized MergeSort : 11.434 ms | Correct: True

Iterative MergeSort : 15.771 ms | Correct: True

Three-way MergeSort : 13.306 ms | Correct: True

==========

Analysis Summary:

==========

Time Complexity:

- Best Case: O(n log n) - consistent performance regardless of input
- Average Case: O(n log n) - always divides array evenly
- Worst Case: O(n log n) - no degradation like QuickSort

Space Complexity:

- Basic MergeSort: O(n) - creates new lists at each level
- In-place MergeSort: O(n) - temporary arrays during merge
- Optimized MergeSort: O(n) - same as in-place but faster
- Iterative MergeSort: O(n) - no recursion stack needed

- Three-way MergeSort: O(n) - similar to basic version

Key Observations:

1. MergeSort guarantees O(n log n) time complexity in all cases

2. Stable sorting algorithm - preserves the relative order of equal elements

3. Optimized version with insertion sort improves performance on small arrays

4. The iterative version avoids the recursion overhead

5. Three-way version reduces the depth of the recursion tree

6. Better than QuickSort for linked lists and guaranteed performance

7. Requires additional space, unlike in-place QuickSort

Advantages over QuickSort:

- Guaranteed O(n log n) performance (no worst case $O(n^2)$)

- Stable sorting (maintains relative order of equal elements)

- Predictable performance for worst-case scenarios

- Better for external sorting (sorting data that doesn't fit in memory)

Disadvantages compared to QuickSort:

- Requires O(n) extra space

- Slower than QuickSort on average for arrays in memory

- More complex to implement a true in-place version

Performance Testing

Example datasets tested:

1. Sorted data

2. Reverse-sorted data
3. Random data

Example Python test code:

```
import random

import time

data = [random.randint(1,10000) for i in range(1000)]

start = time.time()

quick_sort(data)

print("Quick Sort Time:", time.time() - start)


start = time.time()

merge_sort(data)

print("Merge Sort Time:", time.time() - start)
```

Sample Performance Results

Dataset Type	Quick Sort	Merge Sort
Sorted	Slow	Stable
Reverse Sorted	Slow	Stable
Random	Fast	Fast

Observed Differences Between Theory and Practice

Although theoretical analysis suggests both algorithms run in O(n log n) time, practical performance may vary due to several factors:

1. Memory usage

Merge Sort requires extra memory for merging arrays.

2. Cache efficiency

Quick Sort performs better in memory because it is an in-place sorting algorithm.

3. Pivot selection

Poor pivot choices can degrade Quick Sort performance.

4. Python overhead

Recursive calls introduce additional overhead in real execution.

GitHub Repository

https://github.com/Gyanvlon/UC-AI

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.